VERSIONS & MODELS REFERENCE

LeafGuard

Release inventory, merge history and model artifacts

Complete record of every LeafGuard release on GitHub, the two merge commits that brought the v1.4.0.0 (Swin) and v1.5.0.0 (JK-TK YOLOv11) codebases into main as versioned backups, and the APK and model artifacts shipped in each release. Prepared as the backup reference before fine-tuning begins.

REPOSITORY

K-dubey09/Plant-disease-detection-system-XAI

MAIN HEAD AFTER BACKUP

71d47cc · versionName 1.5.0.0

DATE

2026-11-04

Table of Contents

1. Overview and Repository State **2**

1.1 Branches and tags on the remote 2

2. GitHub Release Inventory **2**

3. v1.4.0.0 - Swin/PlantVillage Backup **3**

4. v1.5.0.0 - JK-TK YOLOv11 Detector (Phase A) **3**

5. Merge History on main **4**

6. Model Artifact Details **4**

7. Backup Strategy and Recovery Procedure **5**

1. Overview and Repository State

This reference documents the complete release history of the LeafGuard plant-disease detection Android application as it exists on GitHub at the close of the Phase A backup operation. It records every tagged version, the commit each tag points at, the APK and model artifacts published in each GitHub Release, and the two merge commits that brought the v1.4.0.0 (Swin) and v1.5.0.0 (JK-TK YOLOv11) codebases into the main branch so that both are recoverable from a single branch. The repository is `K-dubey09/Plant-disease-detection-system-XAI` on GitHub.

The backup strategy is two-layered. The codebase layer keeps each version reachable from main through explicit merge commits that name the source branch, the source commit hash, the version code, and the model the version ships. The artifact layer keeps each version installable through a GitHub Release that carries both the debug APK and, where the model exceeds Git limits, the raw model file as a standalone asset. Together these mean any past version can be rebuilt from source or reinstalled from its release without depending on a single point of failure.

1.1 Branches and tags on the remote

After the Phase A merge, the remote carries three branches and six annotated tags. The main branch is the integration target and now contains both backup versions through merge commits; the two version branches are preserved verbatim so the exact pre-merge history remains reachable. No branch was deleted or rewritten.

Branch	Tip commit	Role	versionName
`main`	`71d47cc`	Integration target; contains both merges	1.5.0.0
`v1.4.0.0-swin`	`09e8661`	Swin/PlantVillage backup (pre-JK-TK)	1.4.0.0
`v1.5.0.0-jktk`	`373300a`	JK-TK YOLOv11 detector (Phase A)	1.5.0.0

Table 1. Remote branches after the Phase A merge.

2. GitHub Release Inventory

Six releases are live on the repository. The four early releases (v0.2.0 through v1.0.0) document the incremental phase build-up of the application. The two Phase A releases (v1.4.0.0 and v1.5.0.0) are the backup pair the user asked to keep on GitHub: each carries a debug APK, and the v1.5.0.0 release additionally carries the 217.5 MB JK-TK model as a standalone asset because that file exceeds Git 100 MB per-file limit and so cannot live in the codebase. The v1.4.0.0 release now also carries its 8.66 MB Swin model asset, added during this backup pass so that both releases follow the same APK-plus-model shape.

Tag	Release ID	Commit	Theme	Assets (size)
`v0.2.0`	432137038	`2ee2504`	Phase 2 - PIN authentication	debug APK 41.9 MB
`v0.3.0`	432181055	`1527f37`	Phase 3 - Room + Plants CRUD	debug APK 42.0 MB
`v0.4.0`	432218915	`c69dbd0`	Phase 4 - TFLite inference + XAI	debug APK 50.7 MB; release APK 131.1 MB
`v1.0.0`	433350242	`52c700d`	Production release (all 8 phases)	release APK 131.1 MB

Tag	Release ID	Commit	Theme	Assets (size)
v1.4.0.0	434205681	09e8661	Swin/PlantVillage backup (pre-JK-TK)	debug APK 149.5 MB; Swin model 8.66 MB
v1.5.0.0	434210329	373300a	JK-TK YOLOv11 detector (lab + field)	debug APK 268.4 MB; JK-TK model 217.5 MB

Table 2. All six GitHub releases with their tip commit and assets.

3. v1.4.0.0 - Swin/PlantVillage Backup

v1.4.0.0 is the rollback baseline captured immediately before the JK-TK detector was integrated. It uses the Swin-Tiny image classifier converted to FP32 TFLite and trained on the PlantVillage 38-class lab dataset. It is the version that exhibited the Phase A failure modes the user reported: healthy tomato leaves collapsing to strawberry, sweet potato collapsing to tomato (no sweet-potato class exists in the 38-class scheme), and diseased-tomato identification occasionally landing near 36% confidence. Keeping this version on GitHub means the failing baseline remains installable for comparison and as a one-line rollback target if a later model regresses.

Field	Value
Tag	v1.4.0.0 (annotated, points at commit 09e8661)
Branch	v1.4.0.0-swin (preserved on remote)
Commit	09e8661 - "release(v1.4.0.0): Swin/PlantVillage backup, pre-JK-TK"
versionCode / versionName	6 / 1.4.0.0
Release ID	434205681 (published 2026-11-04 17:01 UTC)
Model	Swin-Tiny, FP32 TFLite, asset path app/src/main/assets/models/plant_disease.tflite
Model size	8.66 MB (9,075,516 bytes); SHA-256 28efdfa3...468b58; committed to git
Labels	app/src/main/assets/models/labels.txt - PlantVillage 38-class scheme
APK asset	LeafGuard-v1.4.0.0-swin-debug.apk, 149.5 MB (asset id 710238700)
Model asset	plant_disease_swin_v1.4.0.0.tflite, 8.66 MB (asset id 710499782, added during backup)

Table 3. v1.4.0.0 release and model metadata.

4. v1.5.0.0 - JK-TK YOLOv11 Detector (Phase A)

v1.5.0.0 is the Phase A working baseline. It replaces the Swin softmax head with a JK-TK YOLOv11 detector trained on a mixed lab-plus-field dataset, so the model generalizes to real-world photos instead of collapsing on field input. An UNKNOWN sentinel class lets the UI say it does not recognize a plant rather than force-fitting sweet potato to tomato or healthy tomato to strawberry. The detector runs on an application-scoped coroutine and the TFLite Interpreter is reused across scans, which fixes the crash that occurred when a scan was left mid-way and a new one started from the scan page. The Swin model and labels are retained in the assets so the rollback is a one-line swap in InferenceModule.kt.

Field	Value
Tag	v1.5.0.0 (annotated, points at commit 373300a)
Branch	v1.5.0.0-jktk (preserved on remote)
Commit	373300a - "feat(v1.5.0.0): integrate JK-TK YOLOv11 detector (lab+field), Swin retained as rollback"
versionCode / versionName	7 / 1.5.0.0
Release ID	434210329 (published 2026-11-04 17:05 UTC)
Detector	JK-TK YOLOv11, FP32 TFLite, asset path app/src/main/assets/models/jktk_detector.tflite
Model size	217.5 MiB (228 MB decimal); over Git 100 MB limit, so gitignored and shipped via release asset + inside the APK
Labels	app/src/main/assets/models/labels_jktk.txt - 116 classes (lab + field) with UNKNOWN sentinel
New code	JktkDetector.kt, JktkLabelParser.kt, LeafClassifier.kt routing in InferenceModule.kt
APK asset	LeafGuard-v1.5.0.0-jktk-debug.apk, 268.4 MB (asset id 710251261)
Model asset	jktk_detector.tflite, 217.5 MB (asset id 710252636)

Table 4. v1.5.0.0 release and model metadata.

5. Merge History on main

Both backup codebases were merged into main as explicit non-fast-forward merge commits so the integration is visible in the history and the source branch and commit are named in the commit message. The v1.4.0.0-swin merge was performed first and bumped the version from 1.0.0 to 1.4.0.0; the v1.5.0.0-jktk merge followed and bumped it to 1.5.0.0 while bringing in the full JK-TK integration (12 files, 3,456 insertions). Because v1.5.0.0-jktk descends linearly from v1.4.0.0-swin, both merges were clean with no conflicts.

Merge commit	Source branch	Source commit	Effect
`8c0ae04`	v1.4.0.0-swin	`09e8661`	Brings Swin backup into main; version 1.0.0 -> 1.4.0.0 (versionCode 6)
`71d47cc`	v1.5.0.0-jktk	`373300a`	Brings JK-TK integration into main; version -> 1.5.0.0 (versionCode 7); Swin retained as rollback

Table 5. The two merge commits on main after the Phase A backup.

After both merges the main HEAD is 71d47cc and the working versionName is 1.5.0.0. The v1.4.0.0-swin and v1.5.0.0-jktk branches remain on the remote unchanged, so the exact pre-merge state of either version is always reachable by checking out its branch or tag.

6. Model Artifact Details

The two models that back the two Phase A releases differ sharply in size and in how they are stored. The Swin model is small enough to commit directly: it lives in the repository and every clone of any version that includes it carries the bytes. The JK-TK model is 228 MB of FP32 weights, which is more than twice Git 100 MB per-file hard limit, so it is gitignored and instead published as a release asset and embedded inside the debug APK. The gitignore entry on the v1.5.0.0-jktk branch documents this explicitly and points rebuilders to the release asset or the private Kaggle dataset.

Property	Swin (v1.4.0.0)	JK-TK YOLOv11 (v1.5.0.0)
Asset filename	`plant_disease.tflite`	`jktk_detector.tflite`
Asset path	`app/src/main/assets/models/`	`app/src/main/assets/models/`
Size	8.66 MB (9,075,516 bytes)	217.5 MiB (228 MB decimal)
Format	FP32 TFLite	FP32 TFLite
Architecture	Swin-Tiny (Vision Transformer)	YOLOv11 (single-stage detector)
Training data	PlantVillage, 38 classes, lab only	Mixed lab + field, 116 classes incl. UNKNOWN sentinel
In git?	Yes - committed	No - gitignored (>100 MB)
In release?	Yes - 8.66 MB asset	Yes - 217.5 MB asset
In APK?	Yes - bundled in assets	Yes - bundled in assets
Labels file	`labels.txt (38 classes)`	`labels_jktk.txt (116 classes)`

Table 6. Side-by-side comparison of the two model artifacts.

7. Backup Strategy and Recovery Procedure

Each Phase A version can be recovered in two independent ways. If the codebase is needed, check out the version branch or tag directly; the merge commits on main also name the source commits, so the history is self-describing. If an installable artifact is needed without a rebuild, download the APK from the corresponding GitHub Release. If the model itself is needed for a custom rebuild, download the model asset from the release and drop it into app/src/main/assets/models/ before running ./gradlew assembleDebug.

> **REBUILD V1.4.0.0 (SWIN) FROM SOURCE**
>
> git checkout v1.4.0.0-swin (or git checkout v1.4.0.0). The Swin model plant_disease.tflite is already in the repo, so no extra download is needed. Run ./gradlew assembleDebug to produce the debug APK.

REBUILD V1.5.0.0 (JK-TK) FROM SOURCE

git checkout v1.5.0.0-jktk (or git checkout v1.5.0.0). The jktk_detector.tflite model is gitignored, so download it from the v1.5.0.0 release assets (jktk_detector.tflite, 217.5 MB) and place it at app/src/main/assets/models/jktk_detector.tflite before running ./gradlew assembleDebug. The small labels_jktk.txt and disease_suggestions_jktk.json ARE committed, so no other download is needed.

REINSTALL WITHOUT REBUILDING

Download LeafGuard-v1.4.0.0-swin-debug.apk (149.5 MB) or LeafGuard-v1.5.0.0-jktk-debug.apk (268.4 MB) from the corresponding GitHub Release and install with adb install -r . No source checkout or Gradle build required.

The two backups are deliberately kept as distinct versions rather than collapsed into a single latest release. This means a regression in a future fine-tuned model can be compared against the exact v1.5.0.0 baseline, and the pre-JK-TK Swin baseline remains available for the failure-mode analysis that motivated the move to YOLOv11 in the first place.